Note that the orthogonal projection $P_{N(w_0)}$ is the *least harming linear operation* to remove the linear information captured by W_0 from X, in the sense that among all maximum rank (which is not full, as such transformations are invertible—hence not linearly guarding) projections onto the nullspace of W_0, it carries the least impact on distances. This is so since the image under an orthogonal projection into a subspace is by definition the closest vector in that subspace.

***Iterative* Projection** Projecting the inputs X on the nullspace of a single linear classifier does not suffice for making Z linearly guarded: classifiers can often still be trained to recover z from the projected x with above chance accuracy, as there are often multiple linear directions (hyperplanes) that can partially capture a relation in multidimensional space. This can be remedied with an iterative process: After obtaining $P_{N(W_0)}$, we train classifier W_1 on $P_{N(W_0)}X$, obtain a projection matrix $P_{N(W_1)}$, train a classifier W_2 on $P_{N(W_1)}P_{N(W_0)}X$ and so on, until no classifier W_{m+1} can be trained. We return the guarding projection matrix $P = P_{N(W_m)}P_{N(W_{m-1})}...P_{N(W_0)}$, with the guarding function $g(x) = Px$. Crucially, the ith classifier W_i is trained on the data X after the projection on the nullspaces of classifiers W_0, ..., W_{i-1} and is therefore trained to find separating planes that are independent of the separating planes found by previous classifiers.

In Appendix §A.1 we prove three desired proprieties of INLP: (1) any two protected-attribute classifiers found in INLP are orthogonal (Lemma A.1); (2) while in general the product of projection matrices is not a projection, the product P calculated in INLP is a valid projection (Corollary A.1.2); and (3) it projects any vector to the intersection of the nullspaces of each of the classifiers found in INLP, that is, after n INLP iterations, P is a projection to $N(W_0) \cap N(W_1) \cdots \cap N(W_n)$ (Corollary A.1.3). We further bound the damage P causes to the structure of the space (Lemma A.2). INLP can thus be seen as a linear dimensionality-reduction method, which keeps only those directions in the latent space which are *not* indicative of the protected attribute.

During iterative nullspace projection, the property z becomes increasingly linearly-guarded in Px. For binary protected attributes, each intermediate W_j is a vector, and the nullspace rank is $d-1$. Therefore, after n iterations, if the original rank of X was r, the rank of the projected input $g(X)$ is at least $r - n$. The entire process is formalized in Algorithm 1.

Algorithm 1 Iterative Nullspace Projection (INLP)

Input : (X, Z): a training set of vectors and protected attributes
n: Number of rounds
Result: A projection matrix P
Function `GetProjectionMatrix(`X, Z`)`**:**
  $X_{projected} \leftarrow X$
  $P \leftarrow I$
  for $i \leftarrow 1$ **to** n **do**
    $W_i \leftarrow$ TrainClassifier($X_{projected}, Z$)
    $B_i \leftarrow$ GetNullSpaceBasis(W_i)
    $P_{N(W_i)} \leftarrow B_i B i^T$
    $P \leftarrow P_{N(W_i)}P$
    $X_{projected} \leftarrow P_{N(W_i)}X_{projected}$
  end
  return P

INLP bears similarities to Partial Least Squares (PLS; Geladi and Kowalski (1986); Barker and Rayens (2003)), an established regression method. Both iteratively find directions that correspond to Z: while PLS does so by maximizing covariance with Z (and is thus less suited to classification), INLP learns the directions by training classifiers with an arbitrary loss function. Another difference is that INLP focuses on learning a projection that neutralizes Z, while PLS aims to learn low-dimensional representation of X that keeps information on Z.

Implementation Details A naive implementation of Algorithm 1 is prone to numerical errors, due to the accumulative projection-matrices multiplication $P \leftarrow P_{N(W_i)}P$. To mitigate that, we use the formula of Ben-Israel (2015), which connects the intersection of nullspaces with the projection matrices to the corresponding rowspaces:

$$N(w_1) \cap \cdots \cap N(w_n) = N(P_R(w_1) + \cdots + P_R(w_n)) \quad (1)$$

Where $P_R(W_i)$ is the orthogonal projection matrix to the row-space of a classifier W_i. Accordingly, in practice, we do not multiply $P \leftarrow P_{N(W_i)}P$ but rather collect *rowspace* projection matrices $P_{R(W_i)}$ for each classifier W_i. In place of each input projection $X_{projected} \leftarrow P_{N(W_i)}X_{projected}$, we recalculate $P := P_{N(w_1) \cap ... \cap N(w_i)}$ according to 1, and perform a projection $X_{projected} \leftarrow PX$. Upon

termination, we once again apply 1 to return the final nullspace projection matrix $P_{N(W_1)\cap...\cap N(W_n)}$. The code is publicly available.[2]

5 Application to Fair Classification

The previous section described the INLP method for producing a linearly guarding function g for a set of vectors. We now turn to describe its usage in the context of providing fair classification by a (possibly deep) neural network classifier.

In this setup, we are given, in addition to X and Z also labels Y, and wish to construct a classifier $f : X \rightarrow Y$, while being *fair* with respect to Z. Fairness in classification can be defined in many ways (Hardt et al., 2016; Madras et al., 2019; Zhang et al., 2018). We focus on a notion of fairness by which the predictor f is oblivious to Z when making predictions about Y.

To use linear guardedness in the context of a deep network, recall that a classification network $f(x)$ can be decomposed into an encoder enc followed by a linear layer W: $f(x) = W \cdot enc(x)$, where W is the last layer of the network and enc is the rest of the network. If we can make sure that Z is linearly guarded in the inputs to W, then W will have no knowledge of Z when making its prediction about Y, making the decision process oblivious to Z. Adversarial training methods attempt to achieve such obliviousness by adding an adversarial objective to make $enc(x)$ itself guarding. We take a different approach and add a guarding function *on top of* an already trained enc.

We propose the following procedure. Given a training set X,Y and protected attribute Z, we first train a neural network $f = W \cdot enc(X)$ to best predict Y. This results in an encoder that extracts effective features from X for predicting Y. We then consider the vectors $enc(X)$, and use the INLP method to produce a linear guarding function g that guards Z in $enc(X)$. At this point, we can use the classifier $W \cdot g(enc(x))$ to produce oblivious decisions, however by introducing g (which is lower rank than $enc(x)$) we may have harmed Ws performance. We therefore freeze the network and fine-tune only W to predict Y from $g(enc(x))$, producing the final fair classifier $f'(x) = W' \cdot g(enc(x))$. Notice that W' only sees vectors which are linearly guarded for Z during its training, and therefore cannot take Z into account when making its predictions, ensuring fair classification.

We note that our notion of fairness by obliviousness does not, in the general case, correspond to other fairness metrics, such as equality of odds or of opportunity. It does, however, *correlate* with fairness metrics, as we demonstrate empirically.

Further refinement. Guardedness is a property that holds in expectation over an entire dataset. For example, when considering a dataset of individuals from certain professions (as we do in §6.3), it is possible that the entire dataset is guarded for gender, yet if we consider only a subset of individuals (say, only nurses), we may still be able to recover gender with above majority accuracy, in that sub-population. As fairness metrics are often concerned with classification behavior also within groups, we propose the following refinement to the algorithm, which we use in the experiments in §6.2 and §6.3: in each iteration, we train a classifier to predict the protected attribute not on the entire training set, but only on the training examples belonging to a single (randomly chosen) main-task class (e.g. profession). By doing so, we push the protected attribute to be linearly guarded in the examples belonging to each of the main-task labels.

6 Experiments and Analysis

6.1 "Debiasing" Word Embeddings

In the first set of experiments, we evaluate the INLP method in its ability to debias word embeddings (Bolukbasi et al., 2016). After "debiasing" the embeddings, we repeat the set of diagnostic experiments of Gonen and Goldberg (2019).

Data. Our debiasing targets are the uncased version of GloVe word embeddings (Zhao et al., 2018), after limiting the vocabulary to the 150,000 most common words. To obtain labeled data X,Z for this classifier, we use the 7,500 most male-biased and 7,500 most female-biased words (as measured by the projection on the $\overrightarrow{he} - \overrightarrow{she}$ direction), as well as 7,500 neutral vectors, with a small component (smaller than 0.03) in the gender direction. The data is randomly divided into a test set (30%), and training and development sets (70%, further divided into 70% training and 30% development examples).

Procedure We use a L_2-regularized SVM classifier (Hearst et al., 1998) trained to discriminate

[2] https://github.com/Shaul1321/nullspace_projection

between the 3 classes: male-biased, female-biased and neutral. We run Algorithm 1 for 35 iterations.

6.1.1 Results

Classification. Initially, a linear SVM classifier perfectly discriminates between the two genders (100% accuracy). The accuracy drops to 49.3% following INLP. To measure to what extent gender is still encoded in a *nonlinear* way, we train a 1-layer ReLU-activation MLP. The MLP recovers gender with accuracy of 85.0%. This is expected, as the INLP method is only meant to achieve *linear guarding*[3].

Human-selected vs. Learned Directions. Our method differs from the common projection-based approach by two main factors: the numbers of directions we remove, and the fact that those directions are learned iteratively from data. Perhaps the benefit is purely due to removing more directions? We compare the ability to linearly classify words by gender bias after removing 10 directions by our method (running Algorithm 1 for 10 iterations) with the ability to do so after removing 10 manually-chosen directions defined by the difference vectors between gendered pairs [4]. INLP-based debiasing results in a very substantial drop in classification accuracy (54.4%), while the removal of the predefined directions only moderately decreases accuracy (80.7%). This shows that data-driven identification of gender-directions outperforms manually selected directions: there are many subtle ways in which gender is encoded, which are hard for people to imagine.

Discussion. Both the previous method and our method start with the main gender-direction of $\overrightarrow{he} - \overrightarrow{she}$. However, while previous attempts take this *direction* as the information that needs to be neutralized, our method instead considers the *labeling* induced by this gender direction, and then iteratively finds and neutralizes directions that correlate with this labeling. It is likely that the $\overrightarrow{he} - \overrightarrow{she}$ direction is one of the first to be removed, but we then go on and *learn* a set of other directions that correlate with the same labeling and which are predictive of it to some degree, neutralizing each of them in turn. Compared to the 10 manually identified gender-directions from Bolukbasi et al. (2016), it is likely that our learned directions capture a much more diverse and subtle set of gender clues in the embedding space.

Effect of debiasing on the embedding space. In appendix §A.2 we provide a list of 40 random words and their closest neighbors, before and after INLP, showing that INLP doesn't significantly damage the representation space that encodes lexical semantics. We also include a short analysis of the influence on a specific subset of inherently gendered words: gendered surnames (Appendix §A.4).

Additionally, we perform a semantic evaluation of the debiased embeddings using multiple word similarity datasets (e.g. SimLex-999 (Hill et al., 2015)). We find large improvements in the quality of the embeddings after the projection (e.g. on SimLex-999 the correlation with human judgements improves from 0.373 to 0.489) and we elaborate more on these findings in Appendix §A.3.

Clustering. Figure 1 shows t-SNE (Maaten and Hinton, 2008) projections of the 2,000 most female-biased and 2,000 most male-biased words, originally and after $t = 3$, $t = 18$ and $t = 35$ projection steps. The results clearly demonstrate that the classes are no longer linearly separable: this behavior is qualitatively different from previous word vector debiasing methods, which were shown to maintain much of the proximity between female and male-biased vectors (Gonen and Goldberg, 2019). To quantify the difference, we perform K-means clustering to $K = 2$ clusters on the vectors, and calculate the V-measure (Rosenberg and Hirschberg, 2007) which assesses the degree of overlap between the two clusters and the gender groups. For the t-SNE projected vectors, the measure drops from 83.88% overlap originally, to 0.44% following the projection; and for the original space, the measure drops from 100% to 0.31%.

WEAT. While our method does not guarantee attenuating the bias-by-neighbors phenomena that is discussed in Gonen and Goldberg (2019), it is still valuable to quantify to what extent it does mitigate this phenomenon. We repeat the Word Embedding Association Test (WEAT) from Caliskan et al. (2017) which aims to measure the association in vector space between male and female concepts and stereotypically male

[3] Interestingly, RBF-kernel SVM (used by Gonen and Goldberg (2019)) achieves random accuracy.

[4] We use the following pairs, taken from Bolukbasi et al. (2016): ("woman", "man"), ("girl", "boy"), ("she", "he"), ("mother", "father"), ("daughter", "son"), ("gal", "guy"), ("female", "male"), ("her", "his"), ("herself", "himself"), ("mary", "john").